

National University of Singapore
School of Computing
CS1101S: Programming Methodology
Semester I, 2026/2027

S7

Iterative Processes and Symbolic Computation

Problems:

1. Consider the following “fast exponentiation” function for raising a number to the power of a non-negative integer, following SICPy 1.2.4.

```
is_even = lambda x: x % 2 == 0
square = lambda x: x * x

def fast_expt(b, n):
    return (1
            if n == 0
            else square(fast_expt(b, n / 2))
            if is_even(n)
            else b * fast_expt(b, n - 1))
```

The function `fast_expt` gives rise to a recursive process. Write a function `fast_expt_iter` that uses the same method but gives rise to an iterative process.

Try this first using an accumulator idea. Use `display` in `square` to see in what way numbers get squared. Discuss why the accumulator idea works in this case.

Then use CPS to get exactly the same square operations as in `fast_expt`.

2. A *boolean expression tree* is either `True` or `False` or a list with three elements, the first being either `&&` or `||` and the other two being themselves boolean expression trees.

Example:

```
my_exp = llist("&&",
               llist("||", True, False),
               llist("&&", True, False))
```

The boolean expression tree `my_exp` represents the boolean expression

```
(True or False) and (True and False)
```

which evaluates to `False` in Python. Write a function `eval_boolean` that takes as argument a boolean expression tree and returns the boolean value of the expression.

Add boolean negation to the definition of a boolean expression tree and extend your function `eval_boolean` accordingly.

